# HRMS Microservices

Development Tracker · Spring Boot · Ordered by Build Priority

| Microservices | Total DB Tables | Total Endpoints | Stack |
|---|---|---|---|
| 10 | 83 | 147 | Java / Spring Boot |

## #1 HR Admin Service · Port 8080 · Module: hr-admin-service

**Complexity:** MEDIUM | **Why now:** Must be built first. Owns all lookup data, user accounts, roles and audit logging consumed by every other service.

### DB Tables

| # | Table Name | Done? |
|---|---|---|
| 1 | `lookup_categories` | ✅ |
| 2 | `lookup_values` | ✅ |
| 3 | `users` | ✅ |
| 4 | `roles` | ✅ |
| 5 | `audit_log` | ❌ |
| 6 | `system_config` | ❌ |

### API Endpoints

| # | Endpoint | Done? |
|---|---|---|
| 1 | `POST /api/admin/lookup/categories` | ☐ |
| 2 | `GET /api/admin/lookup/categories` | ☐ |
| 3 | `PUT /api/admin/lookup/categories/{id}` | ☐ |
| 4 | `DELETE /api/admin/lookup/categories/{id}` | ☐ |
| 5 | `POST /api/admin/lookup/values` | ☐ |
| 6 | `GET /api/admin/lookup/values` | ☐ |
| 7 | `GET /api/admin/lookup/values/{categoryCode}` | ☐ |
| 8 | `PUT /api/admin/lookup/values/{id}` | ☐ |
| 9 | `DELETE /api/admin/lookup/values/{id}` | ☐ |
| 10 | `POST /api/admin/users` | ☐ |
| 11 | `GET /api/admin/users` | ☐ |
| 12 | `GET /api/admin/users/{id}` | ☐ |
| 13 | `PUT /api/admin/users/{id}` | ☐ |
| 14 | `PUT /api/admin/users/{id}/reset-password` | ☐ |
| 15 | `DELETE /api/admin/users/{id}` | ☐ |

| | | |
|---|---|---|
| 16 | POST /api/admin/roles | ☐ |
| 17 | GET /api/admin/roles | ☐ |
| 18 | PUT /api/admin/roles/{id} | ☐ |
| 19 | POST /api/admin/auth/login | ☐ |
| 20 | POST /api/admin/auth/logout | ☐ |
| 21 | POST /api/admin/auth/refresh | ☐ |
| 22 | GET /api/admin/audit-log | ☐ |
| 23 | GET /api/admin/audit-log/{entityType}/{entityId} | ☐ |
| 24 | GET /api/admin/system-config | ☐ |
| 25 | PUT /api/admin/system-config/{key} | ☐ |

### Key Relationships / Foreign Keys

| # | Relationship |
|---|---|
| 1 | lookup_values → lookup_categories (Many-to-One) |
| 2 | users → roles (Many-to-One) |
| 3 | audit_log → users (Many-to-One, who performed action) |
| 4 | system_config — standalone key-value config table |
| 5 | All other services → lookup_values (via Feign Client by categoryCode) |
| 6 | All other services → audit_log (publish events to Admin Service) |

### Dev Notes & Tips (Spring Boot)

| | |
|---|---|
| 1. | Build this FIRST — every service needs lookups and auth before anything else |
| 2. | Seed lookup_categories and lookup_values on startup (STATE, DEPARTMENT, COMPANY, GENDER, etc.) |
| 3. | Implement JWT here — issue tokens with user ID + roles, validate in all other services via Spring Security filter |
| 4. | Use Spring Cloud Gateway or a shared auth filter so every service validates JWT against this service |
| 5. | audit_log should accept events from all services via a REST endpoint or Kafka topic |
| 6. | Set up AOP in each service to auto-publish to audit_log on every write operation |
| 7. | system_config can store things like max_leave_days, probation_period_months, company name etc. |
| 8. | Use @Cacheable on lookup endpoints — they're read-heavy and rarely change |

## #2 Core HR Service · Port 8081 · Module: core-hr-service

**Complexity: HIGH** | **Why now:** Foundation of all HR data. All microservices depend on employees, departments and positions. Depends on HR Admin for lookups and auth.

### DB Tables

| # | Table Name | Done? |
|---|---|---|
| 1 | employees | ☐ |

| | | |
|---|---|---|
| 2 | departments | ☐ |
| 3 | positions | ☐ |
| 4 | jobs | ☐ |
| 5 | qualifications | ☐ |
| 6 | position_qualifications | ☐ |
| 7 | kpis | ☐ |
| 8 | job_kpis | ☐ |
| 9 | position_kpis | ☐ |
| 10 | banks | ☐ |
| 11 | bsb_directory | ☐ |
| 12 | course_service_providers | ☐ |
| 13 | documents | ☐ |
| 14 | employee_bank_details | ☐ |
| 15 | employee_next_of_kin | ☐ |
| 16 | interview_questions | ☐ |
| 17 | notifications | ☐ |

**API Endpoints**

| # | Endpoint | Done? |
|---|---|---|
| 1 | POST /api/employees | ☐ |
| 2 | GET /api/employees | ☐ |
| 3 | GET /api/employees/{id} | ☐ |
| 4 | PUT /api/employees/{id} | ☐ |
| 5 | DELETE /api/employees/{id} | ☐ |
| 6 | GET /api/departments | ☐ |
| 7 | POST /api/departments | ☐ |
| 8 | GET /api/positions | ☐ |
| 9 | POST /api/positions | ☐ |
| 10 | GET /api/jobs | ☐ |
| 11 | POST /api/jobs | ☐ |
| 12 | GET /api/employees/{id}/bank-details | ☐ |
| 13 | POST /api/employees/{id}/bank-details | ☐ |
| 14 | GET /api/employees/{id}/next-of-kin | ☐ |
| 15 | POST /api/employees/{id}/next-of-kin | ☐ |
| 16 | POST /api/employees/{id}/documents | ☐ |
| 17 | GET /api/employees/{id}/documents | ☐ |

| | | | |
|---|---|---|---|
| 18 | GET | /api/notifications/{employeeId} | ☐ |
| 19 | POST | /api/notifications | ☐ |
| 20 | GET | /api/kpis | ☐ |
| 21 | POST | /api/kpis | ☐ |
| 22 | GET | /api/qualifications | ☐ |
| 23 | POST | /api/qualifications | ☐ |

### Key Relationships / Foreign Keys

| # | Relationship |
|---|---|
| 1 | employees → departments (Many-to-One) |
| 2 | employees → positions (Many-to-One) |
| 3 | employees → lookup_values (FK to HR Admin — gender, status, etc.) |
| 4 | positions → jobs (Many-to-One) |
| 5 | position_qualifications → positions + qualifications (Join) |
| 6 | job_kpis → jobs + kpis (Join) |
| 7 | position_kpis → positions + kpis (Join) |
| 8 | employee_bank_details → employees (One-to-One) |
| 9 | employee_next_of_kin → employees (One-to-Many) |
| 10 | documents → employees (Many-to-One) |
| 11 | notifications → employees (Many-to-One) |

### Dev Notes & Tips (Spring Boot)

| | |
|---|---|
| 1. | All status/type fields (e.g. employee status, gender, marital status) should use lookup_values from HR Admin |
| 2. | Create a shared EmployeeDTO — other services will call GET /api/employees/{id} via Feign Client constantly |
| 3. | Publish a Spring Event or Kafka message when a new employee is created — Onboarding listens to this |
| 4. | Use Flyway or Liquibase for DB migrations from day one |
| 5. | departments and positions are master data — seed them from lookup_values where applicable |
| 6. | notifications table is written to by all services but owned here — expose a POST endpoint others call |

## #3 Recruitment Service · Port 8082 · Module: recruitment-service

**Complexity: HIGH** | **Why now:** Comes before onboarding — recruits become employees. Must exist before Core HR is fully populated.

### DB Tables

| # | Table Name | Done? |
|---|---|---|
| 1 | advertisements | ☐ |
| 2 | cv_applicants | ☐ |
| 3 | interview_interviewers | ☐ |

| | | |
|---|---|---|
| 4 | `interview_responses` | ☐ |
| 5 | `interview_rounds` | ☐ |
| 6 | `interview_schedules` | ☐ |
| 7 | `job_offers` | ☐ |
| 8 | `offer_acceptance_records` | ☐ |
| 9 | `recruitment_requests` | ☐ |
| 10 | `request_approvals` | ☐ |

## API Endpoints

| # | Endpoint | Done? |
|---|---|---|
| 1 | `POST /api/recruitment/requests` | ☐ |
| 2 | `GET /api/recruitment/requests` | ☐ |
| 3 | `PUT /api/recruitment/requests/{id}/approve` | ☐ |
| 4 | `POST /api/recruitment/advertisements` | ☐ |
| 5 | `GET /api/recruitment/advertisements` | ☐ |
| 6 | `POST /api/recruitment/applicants` | ☐ |
| 7 | `GET /api/recruitment/applicants` | ☐ |
| 8 | `GET /api/recruitment/applicants/{id}` | ☐ |
| 9 | `POST /api/recruitment/interviews/schedule` | ☐ |
| 10 | `GET /api/recruitment/interviews/{applicantId}` | ☐ |
| 11 | `POST /api/recruitment/interviews/{id}/response` | ☐ |
| 12 | `POST /api/recruitment/offers` | ☐ |
| 13 | `PUT /api/recruitment/offers/{id}/accept` | ☐ |
| 14 | `PUT /api/recruitment/offers/{id}/reject` | ☐ |

## Key Relationships / Foreign Keys

| # | Relationship |
|---|---|
| 1 | recruitment_requests → positions (FK to Core HR) |
| 2 | recruitment_requests → departments (FK to Core HR) |
| 3 | advertisements → recruitment_requests (Many-to-One) |
| 4 | cv_applicants → advertisements (Many-to-One) |
| 5 | interview_rounds → recruitment_requests (Many-to-One) |
| 6 | interview_schedules → interview_rounds + cv_applicants (Join) |
| 7 | interview_interviewers → interview_schedules + employees (Join) |
| 8 | interview_responses → interview_schedules (Many-to-One) |
| 9 | job_offers → cv_applicants (One-to-One) |
| 10 | offer_acceptance_records → job_offers (One-to-One) |

| Dev Notes & Tips (Spring Boot) | |
|---|---|
| 1. | Trigger employee creation in Core HR when offer_acceptance_records is created |
| 2. | Use Spring Events or Kafka to notify Core HR when a candidate accepts an offer |
| 3. | interview_questions table lives in Core HR — fetch via Feign Client |
| 4. | Implement approval workflow using a simple state machine (PENDING → APPROVED → REJECTED) |
| 5. | Store CV documents via Core HR documents table or an S3 bucket reference |

## #4 Onboarding Service · Port 8083 · Module: onboarding-service

**Complexity:** MEDIUM | **Why now:** Triggered after recruitment when a new hire accepts an offer. Depends on Core HR employee records existing.

### DB Tables

| # | Table Name | Done? |
|---|---|---|
| 1 | `finance_action_lists` | ☐ |
| 2 | `finance_cellphone_details` | ☐ |
| 3 | `induction_checklist_items` | ☐ |
| 4 | `induction_checklists` | ☐ |
| 5 | `induction_training_items` | ☐ |
| 6 | `induction_trainings` | ☐ |
| 7 | `it_hardware` | ☐ |
| 8 | `it_mailing_lists` | ☐ |
| 9 | `it_network_drivers` | ☐ |
| 10 | `it_setups` | ☐ |
| 11 | `it_software` | ☐ |
| 12 | `onboarding_processes` | ☐ |
| 13 | `payroll_setups` | ☐ |

### API Endpoints

| # | Endpoint | Done? |
|---|---|---|
| 1 | `POST /api/onboarding/processes` | ☐ |
| 2 | `GET /api/onboarding/processes/{employeeId}` | ☐ |
| 3 | `PUT /api/onboarding/processes/{id}/status` | ☐ |
| 4 | `GET /api/onboarding/checklists` | ☐ |
| 5 | `POST /api/onboarding/checklists` | ☐ |
| 6 | `PUT /api/onboarding/checklists/{id}/items/{itemId}/complete` | ☐ |
| 7 | `POST /api/onboarding/it/setup` | ☐ |

| | | | |
|---|---|---|---|
| 8 | GET | /api/onboarding/it/hardware/{employeeId} | ☐ |
| 9 | POST | /api/onboarding/it/software | ☐ |
| 10 | POST | /api/onboarding/finance/cellphone | ☐ |
| 11 | POST | /api/onboarding/payroll/setup | ☐ |
| 12 | GET | /api/onboarding/trainings/{employeeId} | ☐ |

### Key Relationships / Foreign Keys

| # | Relationship |
|---|---|
| 1 | onboarding_processes → employees (FK to Core HR) |
| 2 | induction_checklists → onboarding_processes (Many-to-One) |
| 3 | induction_checklist_items → induction_checklists (Many-to-One) |
| 4 | induction_trainings → onboarding_processes (Many-to-One) |
| 5 | induction_training_items → induction_trainings (Many-to-One) |
| 6 | it_setups → employees (FK to Core HR) |
| 7 | it_hardware → it_setups (Many-to-One) |
| 8 | it_software → it_setups (Many-to-One) |
| 9 | payroll_setups → employees (One-to-One) |
| 10 | finance_cellphone_details → employees (One-to-One) |

### Dev Notes & Tips (Spring Boot)

| | |
|---|---|
| 1. | Auto-trigger onboarding_processes creation when Core HR creates a new employee |
| 2. | Use a checklist template system — create default checklists per department/role |
| 3. | it_setups should trigger email notifications to the IT team via notifications in Core HR |
| 4. | payroll_setups should integrate with your payroll provider API (e.g. Xero, MyPay) |
| 5. | Track completion percentage on onboarding_processes for dashboard reporting |

## #5 Leave Management Service · Port 8084 · Module: leave-service

**Complexity:** MEDIUM | **Why now:** Core employee data must exist. Independent of recruitment/onboarding but employees need to be in the system.

### DB Tables

| # | Table Name | Done? |
|---|---|---|
| 1 | leave_approvals | ☐ |
| 2 | leave_balances | ☐ |
| 3 | leave_requests | ☐ |
| 4 | leave_types | ☐ |
| 5 | public_holidays | ☐ |

### API Endpoints

| # | Endpoint | Done? |
|---|---|---|

| | | | |
|---|---|---|---|
| 1 | GET | /api/leave/types | ☐ |
| 2 | POST | /api/leave/types | ☐ |
| 3 | GET | /api/leave/balances/{employeeId} | ☐ |
| 4 | POST | /api/leave/balances/initialize | ☐ |
| 5 | POST | /api/leave/requests | ☐ |
| 6 | GET | /api/leave/requests/{employeeId} | ☐ |
| 7 | GET | /api/leave/requests/pending | ☐ |
| 8 | PUT | /api/leave/requests/{id}/approve | ☐ |
| 9 | PUT | /api/leave/requests/{id}/reject | ☐ |
| 10 | PUT | /api/leave/requests/{id}/cancel | ☐ |
| 11 | GET | /api/leave/public-holidays | ☐ |
| 12 | POST | /api/leave/public-holidays | ☐ |

**Key Relationships / Foreign Keys**

| # | Relationship |
|---|---|
| 1 | leave_requests → employees (FK to Core HR) |
| 2 | leave_requests → leave_types (Many-to-One) |
| 3 | leave_approvals → leave_requests (One-to-One) |
| 4 | leave_approvals → employees/manager (FK to Core HR) |
| 5 | leave_balances → employees + leave_types (Composite FK) |

**Dev Notes & Tips (Spring Boot)**

| | |
|---|---|
| 1. | Initialize leave_balances when a new employee is created (listen to Core HR event) |
| 2. | Deduct leave_balances automatically when leave_approvals status = APPROVED |
| 3. | public_holidays should be loaded per country/state — consider seeding Malaysian holidays |
| 4. | Implement a yearly reset job using @Scheduled to reset annual leave balances |
| 5. | Validate leave dates against public_holidays before approving |

## #6 Appraisal Service · Port 8085 · Module: appraisal-service

**Complexity:** MEDIUM | **Why now:** Depends on Core HR for employees, KPIs and positions. Should come after employees are onboarded.

**DB Tables**

| # | Table Name | Done? |
|---|---|---|
| 1 | appraisal_kpi_scores | ☐ |
| 2 | appraisal_schedules | ☐ |
| 3 | appraisals | ☐ |

**API Endpoints**

| # | Endpoint | Done? |
|---|---|---|
| 1 | `POST /api/appraisals/schedules` | ☐ |
| 2 | `GET /api/appraisals/schedules` | ☐ |
| 3 | `GET /api/appraisals/schedules/{id}` | ☐ |
| 4 | `POST /api/appraisals` | ☐ |
| 5 | `GET /api/appraisals/{employeeId}` | ☐ |
| 6 | `GET /api/appraisals/{id}` | ☐ |
| 7 | `PUT /api/appraisals/{id}/submit` | ☐ |
| 8 | `PUT /api/appraisals/{id}/approve` | ☐ |
| 9 | `POST /api/appraisals/{id}/kpi-scores` | ☐ |
| 10 | `GET /api/appraisals/{id}/kpi-scores` | ☐ |

### Key Relationships / Foreign Keys

| # | Relationship |
|---|---|
| 1 | appraisals → employees (FK to Core HR) |
| 2 | appraisals → appraisal_schedules (Many-to-One) |
| 3 | appraisal_kpi_scores → appraisals (Many-to-One) |
| 4 | appraisal_kpi_scores → kpis (FK to Core HR) |
| 5 | appraisal_schedules → departments (FK to Core HR, optional) |

### Dev Notes & Tips (Spring Boot)

| | |
|---|---|
| 1. | Fetch KPI list from Core HR via Feign Client when creating appraisal_kpi_scores |
| 2. | Use appraisal_schedules to define the review period (e.g. Q1, Annual) |
| 3. | Implement multi-level review: self-review → manager review → HR approval |
| 4. | Add a computed overall_score field on appraisals based on weighted KPI scores |
| 5. | Trigger notifications to employees when an appraisal schedule is created |

## #7 Training Service · Port 8086 · Module: training-service

**Complexity:** MEDIUM | **Why now:** Depends on Core HR employees and course_service_providers. Can run parallel to Appraisal Service.

### DB Tables

| # | Table Name | Done? |
|---|---|---|
| 1 | `invoice_payments` | ☐ |
| 2 | `training_approvals` | ☐ |
| 3 | `training_invoices` | ☐ |
| 4 | `training_request_courses` | ☐ |
| 5 | `training_requests` | ☐ |

| | | |
|---|---|---|
| 6 | `training_result_details` | ☐ |
| 7 | `training_results` | ☐ |

## API Endpoints

| # | Endpoint | Done? |
|---|---|---|
| 1 | `POST /api/training/requests` | ☐ |
| 2 | `GET /api/training/requests/{employeeId}` | ☐ |
| 3 | `GET /api/training/requests/pending` | ☐ |
| 4 | `PUT /api/training/requests/{id}/approve` | ☐ |
| 5 | `PUT /api/training/requests/{id}/reject` | ☐ |
| 6 | `GET /api/training/requests/{id}/courses` | ☐ |
| 7 | `POST /api/training/results` | ☐ |
| 8 | `GET /api/training/results/{employeeId}` | ☐ |
| 9 | `GET /api/training/results/{id}/details` | ☐ |
| 10 | `POST /api/training/invoices` | ☐ |
| 11 | `GET /api/training/invoices/{requestId}` | ☐ |
| 12 | `POST /api/training/invoices/{id}/payments` | ☐ |
| 13 | `GET /api/training/invoices/{id}/payments` | ☐ |

## Key Relationships / Foreign Keys

| # | Relationship |
|---|---|
| 1 | training_requests → employees (FK to Core HR) |
| 2 | training_requests → departments (FK to Core HR) |
| 3 | training_request_courses → training_requests (Many-to-One) |
| 4 | training_request_courses → course_service_providers (FK to Core HR) |
| 5 | training_approvals → training_requests (One-to-One) |
| 6 | training_results → training_requests (One-to-One) |
| 7 | training_result_details → training_results (Many-to-One) |
| 8 | training_invoices → training_requests (One-to-One) |
| 9 | invoice_payments → training_invoices (Many-to-One) |

## Dev Notes & Tips (Spring Boot)

| | |
|---|---|
| 1. | course_service_providers is managed in Core HR — fetch it via Feign Client |
| 2. | Link training completions back to position_training_courses in Core HR for compliance tracking |
| 3. | Auto-generate training_invoices when training_approvals status = APPROVED |
| 4. | Track payment status on invoice_payments: PENDING, PARTIAL, PAID |
| 5. | Consider linking training_results to appraisal_kpi_scores for learning KPIs |

# #8 Disciplinary Service · Port 8087 · Module: disciplinary-service

**Complexity: LOW** | **Why now:** Requires employees to exist in Core HR. Relatively standalone — no downstream dependencies.

## DB Tables

| # | Table Name | Done? |
|---|---|---|
| 1 | `counselling_reviews` | ☐ |
| 2 | `counselling_sessions` | ☐ |
| 3 | `disciplinary_actions` | ☐ |
| 4 | `disciplinary_documents` | ☐ |
| 5 | `disciplinary_outcomes` | ☐ |
| 6 | `hr_disciplinary_advice` | ☐ |

## API Endpoints

| # | Endpoint | Done? |
|---|---|---|
| 1 | `POST /api/disciplinary/actions` | ☐ |
| 2 | `GET /api/disciplinary/actions/{employeeId}` | ☐ |
| 3 | `GET /api/disciplinary/actions/{id}` | ☐ |
| 4 | `PUT /api/disciplinary/actions/{id}/status` | ☐ |
| 5 | `POST /api/disciplinary/actions/{id}/documents` | ☐ |
| 6 | `GET /api/disciplinary/actions/{id}/documents` | ☐ |
| 7 | `POST /api/disciplinary/counselling/sessions` | ☐ |
| 8 | `GET /api/disciplinary/counselling/{employeeId}` | ☐ |
| 9 | `POST /api/disciplinary/counselling/{id}/reviews` | ☐ |
| 10 | `POST /api/disciplinary/outcomes` | ☐ |
| 11 | `GET /api/disciplinary/outcomes/{actionId}` | ☐ |
| 12 | `POST /api/disciplinary/hr-advice` | ☐ |

## Key Relationships / Foreign Keys

| # | Relationship |
|---|---|
| 1 | disciplinary_actions → employees (FK to Core HR) |
| 2 | disciplinary_actions → employees (FK — raised_by, Core HR) |
| 3 | disciplinary_documents → disciplinary_actions (Many-to-One) |
| 4 | disciplinary_outcomes → disciplinary_actions (One-to-One) |
| 5 | counselling_sessions → employees (FK to Core HR) |
| 6 | counselling_reviews → counselling_sessions (Many-to-One) |
| 7 | hr_disciplinary_advice → disciplinary_actions (Many-to-One) |

| Dev Notes & Tips (Spring Boot) | |
|---|---|
| 1. | Disciplinary records are sensitive — enforce strict role-based access (HR_ADMIN only) |
| 2. | Store disciplinary_documents as references to Core HR documents table |
| 3. | Implement a status flow: INITIATED → INVESTIGATION → HEARING → OUTCOME |
| 4. | Link disciplinary_outcomes back to employee record for history tracking |
| 5. | Audit every action using Core HR audit_log via event publishing |

## #9 Separation Service · Port 8088 · Module: separation-service

**Complexity: LOW** | **Why now:** Requires employees to exist. Should be built before Offboarding as it triggers it.

### DB Tables

| # | Table Name | Done? |
|---|---|---|
| 1 | `exit_interview_answers` | ☐ |
| 2 | `exit_interviews` | ☐ |
| 3 | `resignation_acceptances` | ☐ |
| 4 | `retention_offers` | ☐ |
| 5 | `separation_requests` | ☐ |
| 6 | `termination_finals` | ☐ |

### API Endpoints

| # | Endpoint | Done? |
|---|---|---|
| 1 | `POST /api/separation/requests` | ☐ |
| 2 | `GET /api/separation/requests/{employeeId}` | ☐ |
| 3 | `PUT /api/separation/requests/{id}/approve` | ☐ |
| 4 | `PUT /api/separation/requests/{id}/reject` | ☐ |
| 5 | `POST /api/separation/retention-offers` | ☐ |
| 6 | `GET /api/separation/retention-offers/{requestId}` | ☐ |
| 7 | `PUT /api/separation/retention-offers/{id}/respond` | ☐ |
| 8 | `POST /api/separation/resignation/accept` | ☐ |
| 9 | `POST /api/separation/exit-interviews` | ☐ |
| 10 | `POST /api/separation/exit-interviews/{id}/answers` | ☐ |
| 11 | `GET /api/separation/exit-interviews/{employeeId}` | ☐ |
| 12 | `POST /api/separation/termination/final` | ☐ |

| | | |
|---|---|---|
| 13 | `GET /api/separation/termination/{employeeId}` | ☐ |

**Key Relationships / Foreign Keys**

| # | Relationship |
|---|---|
| 1 | separation_requests → employees (FK to Core HR) |
| 2 | retention_offers → separation_requests (One-to-One) |
| 3 | resignation_acceptances → separation_requests (One-to-One) |
| 4 | exit_interviews → separation_requests (One-to-One) |
| 5 | exit_interview_answers → exit_interviews (Many-to-One) |
| 6 | termination_finals → separation_requests (One-to-One) |

**Dev Notes & Tips (Spring Boot)**

| | |
|---|---|
| 1. | On termination_finals creation, trigger Offboarding Service to start offboarding_processes |
| 2. | Use Spring Events or Kafka to notify Offboarding when separation is finalized |
| 3. | Implement retention workflow: resignation → retention offer → employee response → decision |
| 4. | exit_interview_answers should reference a question bank (consider adding to Core HR) |
| 5. | Update employee status to INACTIVE in Core HR when termination_finals is confirmed |

## #10 Offboarding Service · Port 8089 · Module: offboarding-service

**Complexity:** MEDIUM | **Why now:** Last in the lifecycle — triggered by Separation Service. Mirrors Onboarding but in reverse.

**DB Tables**

| # | Table Name | Done? |
|---|---|---|
| 1 | `finance_cellphone_returns` | ☐ |
| 2 | `finance_credit_card_returns` | ☐ |
| 3 | `finance_offboarding_actions` | ☐ |
| 4 | `it_hardware_returns` | ☐ |
| 5 | `it_mailing_list_removals` | ☐ |
| 6 | `it_network_removals` | ☐ |
| 7 | `it_removals` | ☐ |
| 8 | `it_software_removals` | ☐ |
| 9 | `offboarding_processes` | ☐ |
| 10 | `payroll_terminations` | ☐ |

**API Endpoints**

| # | Endpoint | Done? |
|---|---|---|
| 1 | `POST /api/offboarding/processes` | ☐ |
| 2 | `GET /api/offboarding/processes/{employeeId}` | ☐ |

| | | |
|---|---|---|
| 3 | `PUT /api/offboarding/processes/{id}/status` | ☐ |
| 4 | `POST /api/offboarding/it/hardware-returns` | ☐ |
| 5 | `GET /api/offboarding/it/hardware-returns/{employeeId}` | ☐ |
| 6 | `POST /api/offboarding/it/software-removals` | ☐ |
| 7 | `POST /api/offboarding/it/mailing-list-removals` | ☐ |
| 8 | `POST /api/offboarding/it/network-removals` | ☐ |
| 9 | `POST /api/offboarding/finance/cellphone-returns` | ☐ |
| 10 | `POST /api/offboarding/finance/credit-card-returns` | ☐ |
| 11 | `POST /api/offboarding/finance/actions` | ☐ |
| 12 | `POST /api/offboarding/payroll/termination` | ☐ |
| 13 | `GET /api/offboarding/payroll/termination/{employeeId}` | ☐ |

## Key Relationships / Foreign Keys

| # | Relationship |
|---|---|
| 1 | offboarding_processes → employees (FK to Core HR) |
| 2 | offboarding_processes → separation_requests (FK to Separation Service) |
| 3 | it_hardware_returns → offboarding_processes + it_hardware (links to Onboarding) |
| 4 | it_software_removals → offboarding_processes (Many-to-One) |
| 5 | it_mailing_list_removals → offboarding_processes (Many-to-One) |
| 6 | it_network_removals → offboarding_processes (Many-to-One) |
| 7 | it_removals → offboarding_processes (Many-to-One) |
| 8 | finance_cellphone_returns → offboarding_processes (One-to-One) |
| 9 | finance_credit_card_returns → offboarding_processes (One-to-One) |
| 10 | payroll_terminations → offboarding_processes (One-to-One) |

## Dev Notes & Tips (Spring Boot)

| | |
|---|---|
| 1. | Mirror Onboarding Service structure — it_hardware_returns should reference it_hardware from Onboarding |
| 2. | Auto-populate offboarding checklist from Onboarding records (what was issued = what to return) |
| 3. | payroll_terminations should call payroll provider API to stop salary processing |
| 4. | Notify IT team automatically via Core HR notifications when offboarding_processes is created |
| 5. | Mark offboarding_processes as COMPLETE only when all sub-tasks (IT, Finance, Payroll) are done |
| 6. | Archive employee data in Core HR after offboarding_processes is fully COMPLETE |